



Resumo Analítico: Segurança da Informação

Criptografia, PKI, TLS, Autenticação e Controlo de Acessos

Baseado em materiais didáticos do ISEL – Instituto Superior de Engenharia de Lisboa

ANÁLISE TÉCNICA 11 DOCUMENTOS

1. Contexto Geral do Tema

Os documentos analisados constituem um corpo de material didático focado nos fundamentos e aplicações práticas da segurança da informação, desde primitivas criptográficas de baixo nível até protocolos complexos de autenticação e autorização na web.

Domínios abordados:

- Criptografia simétrica e assimétrica
- Infraestrutura de Chave Pública (PKI) e certificados X.509
- Segurança na Camada de Transporte (TLS/SSL)
- Autenticação (passwords, HTTP, cookies) e Autorização (RBAC, OAuth 2.0, OpenID Connect)
- Implementação prática em Java (JCA)

2. Problema Central Discutido

Como garantir a confidencialidade, integridade, autenticidade e o controlo de acessos adequado em sistemas de informação distribuídos (particularmente na web) face a um adversário ativo na rede?

Os documentos detalham ameaças como *man-in-the-middle*, ataques de dicionário e *Cross-Site Request Forgery*, apresentando contramedidas técnicas desde a matemática das primitivas até à arquitetura de protocolos.

3. Principais Argumentos Apresentados

3.1 Criptografia Simétrica e Assimétrica

- **Cifra Simétrica:** Mesma chave para cifrar e decifrar. Modos como **CBC** (com IV imprevisível) e **CTR** são recomendados; **ECB** é criticado por revelar padrões visuais (ex.: imagem do pinguim ainda reconhecível).
- **Cifra Autenticada:** Modos como **GCM** (Galois Counter Mode) combinam confidencialidade e autenticidade numa única operação, evitando vulnerabilidades de esquemas separados.
- **Funções de Hash:** Resistência a colisões e à segunda pré-imagem são essenciais. Aplicações incluem verificação de integridade (**sha256sum**) e derivação de passwords (Bcrypt, Argon2).
- **Cifra Assimétrica:** Utiliza pares de chaves (pública/privada). Na prática, adota-se um **esquema híbrido**: RSA/ECDH para negociar uma chave de sessão simétrica (AES), que é usada para a comunicação em massa.

3.2 Infraestrutura de Chave Pública (PKI) e Certificados

- A autenticidade de chaves públicas resolve-se com certificados **X.509** e uma cadeia de confiança que termina numa *Trust Anchor*.
- **Caminho de certificação:** Validação recursiva: *Validar Cert(Alice) → Obter Chave Pública do Emissor (CA Int) → Validar Cert(CA Int) → ... → Trust Anchor (Raiz)*.
- **Extensões críticas:** **keyUsage** (ex.: **keyCertSign** para CAs) e **BasicConstraints** (indica se é CA e profundidade máxima da cadeia).

3.3 TLS/SSL

- Fornece um canal seguro sobre TCP com três propriedades: **Confidencialidade** (cifra), **Integridade** (MAC) e **Autenticação** (certificados).
- **Handshake (baseado em RSA):**
 1. Cliente envia **ClientHello** .
 2. Servidor envia **ServerHello** e **Certificate** .
 3. Cliente gera **pre_master_secret** , cifra com a chave pública do servidor e envia.
 4. Derivação das chaves de sessão.
- **Ameaça real:** Compromisso da **Diginotar em 2011** permitiu emissão de certificados falsos para o Gmail, quebrando o TLS sem quebrar a criptografia subjacente.

3.4 Autenticação e Controlo de Acessos

- **Passwords:** Vulneráveis a ataques de dicionário e *rainbow tables*. Mitigações: uso de **Salt** (torna cada hash único) e funções de derivação lentas como **Bcrypt** e **Argon2** (vencedor do PHC 2015).
- **HTTP e Cookies:** HTTP é *stateless*; *cookies* mantêm sessão. Recomendações de segurança: flags **HttpOnly** , **Secure** e **SameSite=Strict** (proteção contra CSRF).

- **RBAC (Role-Based Access Control):**
 - **Role ≠ Grupo:** Um role é uma coleção de **permissões**, não de utilizadores.
 - Modelos: RBAC₀ (base), RBAC₁ (hierarquia de roles), RBAC₂ (restrições como Separação de Deveres).
 - Limitações: "explosão de roles" e rigidez em situações de emergência.

3.5 OAuth 2.0 e OpenID Connect

- **OAuth 2.0:** Framework para **Autorização** delegada, sem partilhar a password do utilizador.
 - **Fluxo Authorization Code Grant (recomendado):**
 1. Redirecionamento para o Authorization Server.
 2. Autenticação e consentimento do utilizador.
 3. Devolução de um **code** ao Cliente (front-channel).
 4. Troca do **code** por **access_token** diretamente com o Auth Server (back-channel, usando **client_secret**).
- **OpenID Connect:** Camada de **Autenticação** sobre OAuth 2.0. Introduce o **ID Token** (formato **JWT**) com asserções sobre o utilizador autenticado (**sub** , **email** , **iss**).

4. Dados Numéricos, Estatísticas e Evidências Citadas

Evidência / Dado	Contexto	Fonte (PDF)
Compromisso Diginotar (2011)	Emissão de certificado falso para o Gmail, permitindo MITM sem quebrar criptografia.	07.SSL-TLS.pdf
Bcrypt: Custo 12	Número de iterações (saltRounds=12) para tornar o hash lento e resistente a força bruta.	08.Autenticacao.pdf
Argon2 (Parâmetros OWASP)	m=47104 (46 MiB), t=1 , p=1 . Recomendação para hardening de memória contra hardware especializado.	08.Autenticacao.pdf
RSA Key Size	Recomendação de 2048 bits . Dimensões típicas variam de 2^{1023} a 2^{4095} .	05.JCA-com-codigos.pdf
TLS 1.3	Eliminou suporte a RSA para troca de segredo e esquemas sem cifra autenticada (ex.: AES-CBC).	07.SSL-TLS.pdf
SHA-256 Hash Size	256 bits (32 bytes).	03.funcoes-hash.pdf

5. Conceitos Técnicos Explicados de Forma Simplificada

- **ASN.1 / DER:** Linguagem de descrição de dados usada para definir a estrutura binária rigorosa dos certificados X.509.
- **IV (Initialization Vector):** Número usado uma única vez na cifra (ex.: CBC). Funciona como um "sal" para a cifra de blocos; impede que mensagens iguais comecem com o mesmo criptograma.
- **Salt (em Passwords):** Valor aleatório concatenado à password antes do hash, inviabilizando ataques de pré-computação massiva (*rainbow tables*).
- **JWT (JSON Web Token):** String codificada em Base64Url com três partes:
Header.Payload.Signature . Permite transmitir asserções de forma verificável e segura.
- **Trust Anchor:** Certificado auto-assinado de uma Autoridade de Certificação Raiz pré-instalado no sistema operativo ou browser (ex.: "DigiCert Global Root CA").

6. Metodologia e Implementação Prática (Java - JCA)

Os documentos detalham a utilização da **Java Cryptography Architecture (JCA)** com os seguintes princípios:

- **Independência de Algoritmo:** Programar contra `Cipher.getInstance("AES/CBC/PKCS5Padding")` e não contra uma classe concreta.
- **Provider:** Implementação plugável (ex.: SunJCE, BouncyCastle).

Fluxo típico de cifra:

1. **Geração de Chave:** `KeyGenerator` (simétrico) ou `KeyPairGenerator` (assimétrico).
2. **Obtenção da Engine:** `Cipher.getInstance(transformacao)`.
3. **Inicialização:** `cipher.init(modo, chave, parametros)`.
4. **Processamento:** `update()` + `doFinal()`.

Exemplo: Validação de Cadeia de Certificação (PKIX)

```
// 1. Criar CertStore com todos os certificados possíveis
CertStore certStore = CertStore.getInstance("Collection",
                                           new CollectionCertStoreParameters(certCollection));

// 2. Definir seletor para o certificado final a validar
X509CertSelector certSelector = new X509CertSelector();
certSelector.setCertificate(certificacao);

// 3. Construir parâmetros PKIX com Trust Anchors e CertStore
PKIXBuilderParameters params = new PKIXBuilderParameters(trustAnchorSet, certSelector);
params.addCertStore(certStore);

// 4. Construir e validar caminho
CertPathBuilder certPathBuilder = CertPathBuilder.getInstance("PKIX");
CertPathBuilderResult result = certPathBuilder.build(params);
```

7. Conclusões e Implicações Práticas

1. **Segurança em Profundidade:** Não basta cifrar; é necessário autenticar a cifra (GCM), autenticar a chave (Certificados) e autenticar o emissor (Assinatura Digital).
2. **TLS é fundamental, mas a PKI é o elo mais fraco:** Compromissos de CAs (ex.: Diginotar) tornam o TLS vulnerável a MITM.
3. **Armazenamento de Passwords:** Hash simples (SHA256) é insuficiente. A norma atual exige funções com custo de memória e CPU ajustável (**Argon2** ou **Bcrypt**).

4. **Delegação de Acesso (OAuth 2.0):** Nunca se deve pedir a password do utilizador para um serviço de terceiros. O fluxo *Authorization Code* garante que a aplicação cliente nunca vê as credenciais.
5. **RBAC vs. ABAC:** O modelo RBAC tem limitações em contextos dinâmicos, sugerindo a evolução para modelos baseados em atributos (ABAC).

⚠ **Nota sobre ambiguidades:** Alguns PDFs contêm páginas com apenas números (ex.: Pág. 18 de 07.SSL-TLS.pdf), sugerindo slides de transição ou anotações incompletas do orador. O fluxo Implicit do OAuth 2.0 é mencionado mas não detalhado, alinhando-se com as práticas atuais que o descontinuam por razões de segurança.

Lista de Termos-Chave

- **AES** (Advanced Encryption Standard)
- **Argon2** – Função de derivação de chave (PHC 2015)
- **ASN.1** (Abstract Syntax Notation One)
- **Bcrypt** – Função de hash adaptativa para passwords
- **CBC** (Cipher Block Chaining)
- **CertPath** (Certification Path)
- **Cipher Suite** – Conjunto de algoritmos negociados no TLS
- **Cookie** – Mecanismo HTTP para estado no cliente
- **CSRF** (Cross-Site Request Forgery)
- **CTR** (Counter Mode)
- **ECB** (Electronic Codebook)
- **GCM** (Galois Counter Mode)
- **HMAC** (Hash-based Message Authentication Code)
- **IV** (Initialization Vector)
- **JCA** (Java Cryptography Architecture)
- **JWT** (JSON Web Token)
- **KeyStore** – Repositório de chaves e certificados em Java
- **MAC** (Message Authentication Code)
- **OAuth 2.0** – Framework para delegação de autorização
- **OID** (Object Identifier)
- **OpenID Connect** – Camada de identidade sobre OAuth 2.0
- **PKI** (Public Key Infrastructure)
- **PKIX** – Perfil de PKI para a Internet
- **RBAC** (Role-Based Access Control)
- **RSA** – Algoritmo de chave pública
- **Salt** – Valor aleatório para proteção de hashes
- **SHA-256** – Função de hash segura de 256 bits
- **TLS** (Transport Layer Security)
- **Trust Anchor** – Ponto de confiança inicial numa PKI
- **X.509** – Norma para certificados digitais

Documento gerado com base em 11 ficheiros PDF do ISEL – Análise estruturada para fins educacionais.
Resumo elaborado em 10 de abril de 2026.